

实验报告

练习1: 加载应用程序并执行

设计实现过程

在 `load_icode` 函数中，我们需要设置新进程的 `trapframe`，以便在内核返回用户态时，处理器能够正确地跳转到应用程序的入口点，并设置正确的栈指针和特权级。

具体实现如下（位于 `kern/process/proc.c` 的 `load_icode` 函数中）：

```
tf->gpr.sp = USTACKTOP;
tf->epc = elf->e_entry;
tf->status = (read_csr(sstatus) & ~SSTATUS_SPP) | SSTATUS_SPIE;
```

1. 设置栈指针 (`tf->gpr.sp`): 将用户栈指针设置为 `USTACKTOP`。这是用户地址空间中栈的顶部。
2. 设置程序计数器 (`tf->epc`): 将异常程序计数器设置为 ELF 可执行文件的入口地址 (`elf->e_entry`)。当执行 `sret` 指令从内核态返回时，PC 将跳转到这个地址。
3. 设置状态寄存器 (`tf->status`):
 - `read_csr(sstatus) & ~SSTATUS_SPP`: 清除 `SPP` (Supervisor Previous Privilege) 位。`SPP` 位决定了从 S 模式返回时进入的特权级。将其设置为 0 表示返回到 User 模式。
 - `| SSTATUS_SPIE`: 设置 `SPIE` (Supervisor Previous Interrupt Enable) 位。这确保了在返回用户态后，中断是被允许的。

用户态进程执行流程描述

当这个用户态进程被 uCore 选择占用 CPU 执行（RUNNING 态）到具体执行应用程序第一条指令的整个经过如下：

1. 调度器选择: `schedule` 函数从运行队列中选择该进程，并调用 `proc_run`。
2. 上下文切换: `proc_run` 调用 `switch_to` 函数，保存当前进程的上下文（寄存器状态），并恢复新进程的上下文。

3. 内核入口: 新进程的上下文设置其返回地址 (ra) 为 `forkret`。因此, `switch_to` 返回后, 执行流跳转到 `forkret`。
4. 中断返回准备: `forkret` 调用 `forkrets`, 并将当前进程的 `trapframe` 指针作为参数传递。
5. 恢复寄存器: `forkrets` (通常在 `trapentry.S` 中实现) 将 `trapframe` 中的内容恢复到 CPU 寄存器中。这包括通用寄存器、`sepc` (设置为应用程序入口)、`sstatus` (设置为用户模式且开启中断)。
6. 执行 `sret`: 执行 `sret` 指令。CPU 根据 `sstatus.SPP` 切换到用户模式 (U-mode), 根据 `sstatus.SPIE` 恢复中断使能状态, 并将 PC 跳转到 `sepc` 指向的地址 (即应用程序的第一条指令)。
7. 执行应用: 应用程序开始执行。

练习2: 父进程复制自己的内存空间给子进程

设计实现过程

在 `copy_range` 函数中, 我们实现了内存的复制。为了支持 Copy-on-Write (COW) 机制, 我们需要根据 `share` 参数来决定是进行物理页的复制还是共享。

具体实现如下 (位于 `kern/mm/pmm.c` 的 `copy_range` 函数中):

```
if (share)
{
    // COW implementation
    if (perm & PTE_W)
    {
        // 如果页面是可写的, 清除写权限并设置 COW 标志
        uint32_t cow_perm = (perm & ~PTE_W) | PTE_COW;

        // 更新父进程的 PTE
        *ptep = pte_create(page2ppn(page), cow_perm);

        // 为子进程创建 PTE, 共享同一个物理页
        *nptep = pte_create(page2ppn(page), cow_perm);

        // 增加物理页的引用计数
        page_ref_inc(page);
    }
    else
    {
        // 如果页面本身只读, 直接共享, 不需要设置 COW
```

```

        *nptep = pte_create(page2ppn(page), perm);
        page_ref_inc(page);
    }
}
else
{
    // 非共享模式（原有实现）：分配新页并复制内容
    struct Page *npage = alloc_page();
    assert(npage != NULL);
    int ret = 0;
    void *src_kvaddr = page2kva(page);
    void *dst_kvaddr = page2kva(npage);
    memcpy(dst_kvaddr, src_kvaddr, PGSIZE);
    ret = page_insert(to, npage, start, perm);
    assert(ret == 0);
}

```

Copy on Write (COW) 机制设计

概要设计：

Copy-on-Write 的核心思想是推迟物理内存的复制操作，直到真正需要写入时才进行。

1. **Fork 阶段**: 当父进程创建子进程时，不立即复制物理内存。而是将父子进程的虚拟页都映射到同一个物理页上。
2. **权限设置**: 将这些共享页面的页表项 (PTE) 设置为只读，并标记为 **COW** (Copy-on-Write)。
3. **写操作触发**: 当任一进程尝试写入这些只读页面时，CPU 会触发页访问异常 (Page Fault)。
4. **异常处理**:
 - 内核的页错误处理程序 (`do_pgfault`) 捕获异常。
 - 检查异常原因是否为写操作，且该页面是否标记为 COW。
 - 如果是，内核分配一个新的物理页。
 - 将原物理页的内容复制到新物理页。
 - 更新当前进程的页表，将虚拟页映射到这个新物理页，并将权限修改为可写，清除 COW 标志。
 - 减少原物理页的引用计数。
5. **恢复执行**: 异常处理返回，重新执行刚才触发异常的写指令，此时写入将成功。

详细设计：

- **PTE 标志位**: 在 `kern/mm/mmu.h` 中定义 `PTE_COW` (例如使用 RSW 位)。

- 引用计数: 利用 `struct Page` 中的 `ref` 成员来跟踪物理页被多少个进程共享。
- `do_fork`: 调用 `copy_mm -> dup_mmap -> copy_range`。在 `copy_range` 中, 如果 `share=1`, 则执行上述的共享逻辑 (设置只读、设置 COW、增加引用计数)。注意需要刷新 TLB。
- `do_pgfault`: 在 `kern/mm/vmm.c` 中。
 - 当发生写异常 (`error_code & 1`) 时, 检查 PTE。
 - 如果 `*ptep & PTE_COW` 为真:
 - 如果 `page_ref(page) > 1`: 说明还有其他进程共享此页。分配新页, 拷贝内容, 建立新映射 (可写), 原页引用计数减 1。
 - 如果 `page_ref(page) == 1`: 说明只有当前进程引用此页 (其他进程可能已经 COW 分离了, 或者已经退出了)。直接将当前 PTE 修改为可写, 清除 COW 标志, 不需要分配新页。

练习3: 阅读分析源代码

fork/exec/wait/exit 执行流程分析

1. fork:

- 用户态: 调用 `fork()` 库函数, 最终执行 `ecall` 指令陷入内核。
- 内核态: `sys_fork -> do_fork`。
 - 分配 `proc_struct`。
 - 分配内核栈。
 - `copy_mm`: 复制或共享内存空间 (COW 在此发生)。
 - `copy_thread`: 设置子进程的 `trapframe` 和上下文。子进程的 `tf->gpr.a0` 被置为 0 (返回值)。
 - 将子进程加入进程列表, 状态设为 `PROC_RUNNABLE`。
 - 返回子进程 PID 给父进程。
- 返回: 父进程从 `sys_fork` 返回 PID。子进程被调度后, 从 `forkret` 开始, 最终返回 0。

2. exec:

- 用户态: 调用 `exec()` 系列函数, 执行 `ecall`。
- 内核态: `sys_exec -> do_execve`。
 - 检查并释放当前进程的内存空间 (`exit_mmap`, `put_pgdir`, `mm_destroy`)。

- `load_icode`: 加载新的 ELF 二进制文件，建立新的内存映射，设置新的 `trapframe`（如练习1所述）。
- 返回: 不返回原来的程序点，而是返回到新程序的入口点 (`elf->e_entry`) 开始执行。

3. wait:

- 用户态: 调用 `wait()`，执行 `ecall`。
- 内核态: `sys_wait` -> `do_wait`。
 - 查找状态为 `PROC_ZOMBIE` 的子进程。
 - 如果找到，释放子进程剩余的资源（内核栈、`proc_struct`），并获取子进程的退出码。
 - 如果没有 ZOMBIE 子进程但有运行中的子进程，将当前进程状态设为 `PROC_SLEEPING` 并调用 `schedule()` 让出 CPU。
- 返回: 返回子进程 PID 和退出状态。

4. exit:

- 用户态: 调用 `exit()`，执行 `ecall`。
- 内核态: `sys_exit` -> `do_exit`。
 - 释放大部分内存资源 (`exit_mmap`)。
 - 将状态设为 `PROC_ZOMBIE`。
 - 设置退出码。
 - 如果有父进程在等待 (`WT_CHILD`)，唤醒父进程。
 - 将所有子进程过继给 `init` 进程。
 - 调用 `schedule()`，不再返回。

内核态与用户态的交错执行:

程序正常在用户态执行。当需要操作系统服务（如 `fork`）或发生中断/异常（如页错误）时，硬件机制将特权级切换到 S 模式（内核态），跳转到内核的中断处理程序。内核处理完毕后，通过 `sret` 指令恢复上下文并返回用户态。

结果返回:

系统调用的返回值通常存放在寄存器 `a0` 中。内核在处理系统调用时，会将返回值写入当前进程 `trapframe` 的 `a0` 字段。当 `forkrets` 恢复寄存器时，用户程序就能在 `a0` 中看到返回值。

用户态进程执行状态生命周期图



- **PROC_UNINIT**: 进程刚被创建，尚未初始化完成。
- **PROC_RUNNABLE**: 进程已准备好运行，在就绪队列中等待调度。
- **PROC_RUNNING**: 进程正在 CPU 上执行（在 uCore 的简化模型中，RUNNING 状态通常隐含在 RUNNABLE 中，即 `current` 指向的 RUNNABLE 进程即为 RUNNING）。
- **PROC_SLEEPING**: 进程因等待某些事件（如子进程退出、IO）而挂起。
- **PROC_ZOMBIE**: 进程已退出，等待父进程回收资源。

扩展练习 Challenge

用户程序是何时被预先加载到内存中的？

在本次实验中，用户程序是在内核启动加载时，作为内核镜像的一部分被一同加载到物理内存中的。

具体机制分析：

通过查看 `Makefile` 可以发现，内核的链接规则如下：

```
$(kernel): $(KOBJS) $(USER_BINS)
    @echo + ld $@
    $(V)$(LD) $(LDFLAGS) -T tools/kernel.ld -o $@ $(KOBJS) --
format=binary $(USER_BINS) --format=default
```

这里 `$(USER_BINS)` 包含了编译好的用户程序二进制文件。链接器 `ld` 使用 `--format=binary` 选项将这些二进制文件直接链接到内核可执行文件 (`bin/kernel`) 中。链接器会自动为这些二进制文件生成起始地址符号（如 `__binary_obj__user_exit_out_start`）。

因此，当 Bootloader（如 OpenSBI 或 QEMU 的加载器）将内核镜像加载到物理内存时，这些用户程序的二进制代码也随之被加载到了内存中。在 `kern/process/proc.c` 中，通过 `KERNEL_EXECVE` 宏直接引用这些内存地址来获取用户程序的代码。

与我们常用操作系统的加载有何区别，原因是什么？

区别：

- 本次实验中用户程序**静态链接**在内核镜像中，随内核一起加载到内存。它们驻留在物理内存中，不依赖文件系统进行存储和检索。
- 而常用操作系统中 (如 Linux/Windows): 用户程序通常存储在**磁盘**（或其他持久化存储设备）的文件系统中。只有当用户请求执行某个程序（例如调用 `exec` 系统调用）时，操作系统才会通过文件系统驱动程序解析可执行文件（如 ELF 或 PE 格式），将其从磁盘**动态加载**到内存中。通常还会利用**按需分页 (Demand Paging)** 机制，仅在访问特定页面时才从磁盘读取数据。

原因：

1. **简化实验设计**: 在 Lab5 阶段，实验的重点是进程管理（Process Management）和虚拟内存管理（Virtual Memory Management）。此时 uCore 尚未实现完善的文件系统（File System，将在 Lab8 中实现）。为了让用户程序能够运行并测试进程管理功能，将用户程序直接嵌入内核是最简单、最直接的方法，避免了引入文件系统的复杂性。
2. **嵌入式场景**: 这种做法在一些资源受限、没有文件系统的简单嵌入式系统或实时操作系统 (RTOS) 中也是存在的，称为 XIP (Execute In Place) 或直接将应用固化在 Flash 中。